

架构侦察

234 个国际象棋评估架构的扫描研究, 横跨难度、阶段、评估桶与战术主题的同任务对比。

作者	Lennart Axel Conrad (学号: 2025080264)
所属单位	清华大学紫荆书院
指导教师	韩军功教授 (清华大学自动化系)
侦察日期	2026-05-09 至 2026-05-10
任务	puzzle_binary (单正 logit, BCE 损失)
数据集	CRTK 标注的 3 类切分 (约 173k 训练 / 21k 验证 / 21k 测试, FEN 零重叠)
硬件	RTX 3070 (8 GiB) — 单 GPU, 单种子 (42), base 规模
预算	最多 12 epoch, patience 3, 每任务 60 分钟墙时, 仅 CUDA
生成日期	2026-05-11
代码仓库	github.com/LenniAConrad/chess-nn-playground

本工作要做什么

我们在一个小规模二分类任务 (谜题 vs 非谜题) 上比较一系列国际象棋评估架构, 用以识别**国际象棋引擎实际上应该采用哪些结构性先验**。“最佳”由两条维度同时衡量:

- 样本效率** — 在固定训练预算下的测试 PR AUC, 决定每训练样本能换到多少 Elo。
- 推理速度** — 在固定 GPU 上每秒的网络评估次数, 决定每搜索秒能换到多少 Elo。对于在每步走子上跑数千节点的 MCTS 引擎, $2\times$ 的速度优势通常比 30 Elo 的准确率优势更值得。

本侦察 (本报告) 使用 puzzle_binary 作为小规模代理任务; 与引擎相关的最终产出是 i243 提案 (§12), 它将存活下来的先验组合成可在引擎级规模下训练的架构。

摘要

234 个国际象棋评估架构各自被训练一次, 规模较小, 用于谜题检测任务。**175** 个产出可用结果; 50 个因代码错误崩溃 (21%); 9 个超过 60 分钟训练时限 (4%)。

234

已训练架构

175

完成运行

180

进入排行榜

0.8755

最高测试 PR AUC

核心发现

i193_exchange_then_king_dual_stream 以明显优势领先 (**0.8755** 测试 PR AUC, 比第二名高出 +0.014, 且参数预算相同)。其双流架构 — 一条用于战术兑子, 一条用于王安全 — 是整个侦察池中编码相同条件下最大的架构差距, 也是唯一能在经验上将排行榜推高到所有通用主干自然 ~ 0.86 上限之上的架构。

1 数据集与谜题类别定义

侦察任务为 `puzzle_binary`: 将棋局分类为战术谜题 (正类) 或 非谜题 (负类)。底层 CRTK 标签是三分类的, 训练时再折叠为二分类 — 但三分类标签决定了数据集的难度结构, 因此本报告中的所有混淆矩阵与逐类分析仍保留三分类。

<code>fine_label = 0</code>	非谜题 (Non-puzzle) 。从大师对局语料中随机采样的棋局, 即使存在战术也是偶然的。本类为简单负样本。
<code>fine_label = 1</code>	验证过的近谜题 (Verified-near-puzzle) 。被 CRTK 过滤器标记为候选谜题, 但经 Stockfish 验证证明并非唯一解战术 — 第二好的走法分数几乎相同。本类为困难负样本, 是区分强弱架构的关键。
<code>fine_label = 2</code>	谜题 (Puzzle) 。具有唯一战术解的棋局: 单一最佳走法的分数至少比其他选择高若干百厘兵。本类为正样本。

生成这些标签的 **CRTK 流水线**位于 `scripts/data/build_crtk_tagged_splits.py`; 决定哪些 Stockfish 阈值能将候选样本提升为 `fine_label 2` 的契约在 `docs/crtk_export_contract.md` 中说明。每个棋局都带有辅助标签 — `crtk_difficulty`、`crtk_phase`、`crtk_eval_bucket`、`crtk_tactic_motifs` — 这些标签被用来对排行榜进行切片 (见下一节)。

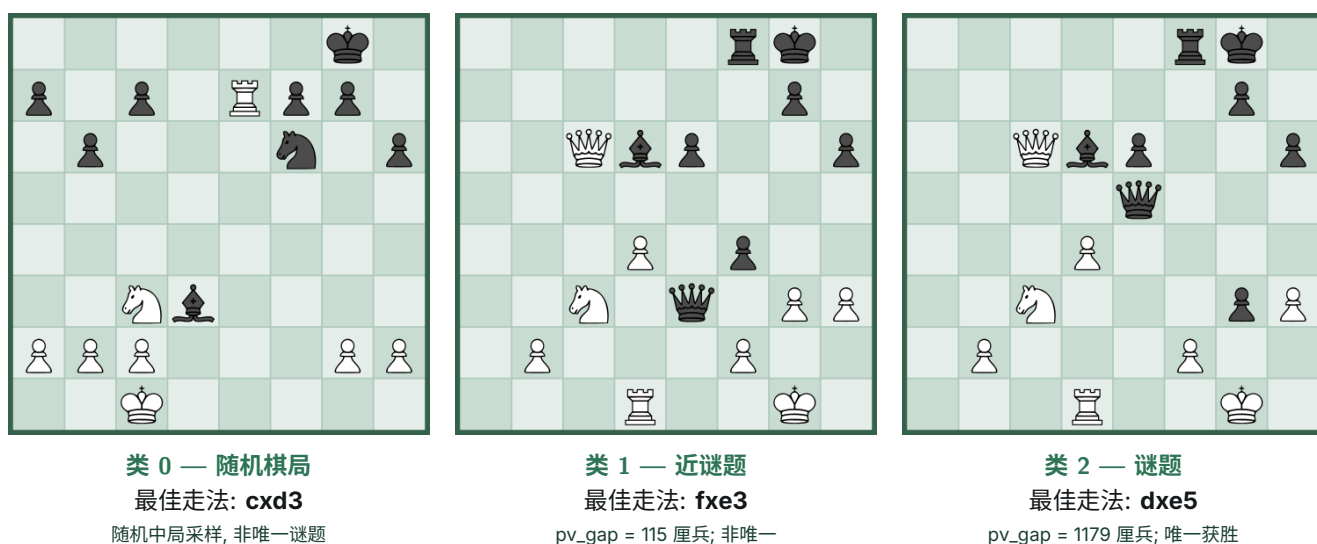


Figure 1: 每个类别选取一个具有代表性的样本, 使用 `crtk fen render` 渲染。三幅图均为白方走子。棋盘只显示原始局面 — 最佳走法以代数记号在每幅图下方报告, 而不是覆盖在棋盘上, 让读者看到网络所看到的画面。**这并非刻意挑选的巧合:** 按 CRTK 的构造方式, 每一行类 1 样本 (近谜题) 都是通过修改某个真正谜题的父行而生成, 所以数据库中每一个近谜题在 CRTK 超语料中都有一个对应的类 2 兄弟样本。在我们的 173k 局面切分中, 约 5% 近谜题的兄弟在采样后保留下来; 上面的类 1 与类 2 棋盘就是这样一对姐妹样本 (CRTK 父节点 `crtk_parent_-1001554678727017229`): 唯一结构差异是类 2 中黑方已经吃入 e5, 使局面变为白方唯一战术胜利 (`dxe5` → 吃后)。这将近谜题与真正谜题成对约束到数据所允许的最紧密程度 — 同一延续, 仅差一步, 而二分标签因唯一应答性质翻转。

三分类结构为何关键

仅靠记住子力数与王安全先验取得高 PR AUC 的模型, 会在二分类排行榜上看起来强劲, 却会在匹配召回近谜题 FP 率上崩塌: 在固定真阳率下, 误将类 1 棋局判为谜题的比例。这一指标在 §7 中区分了真正去检验战术序列的架构。

2 总排行榜

base 规模 12 epoch 训练后的 puzzle_binary 任务测试 PR AUC。 `lc0_bt4_112` 与 `simple_18` 标识输入编码。

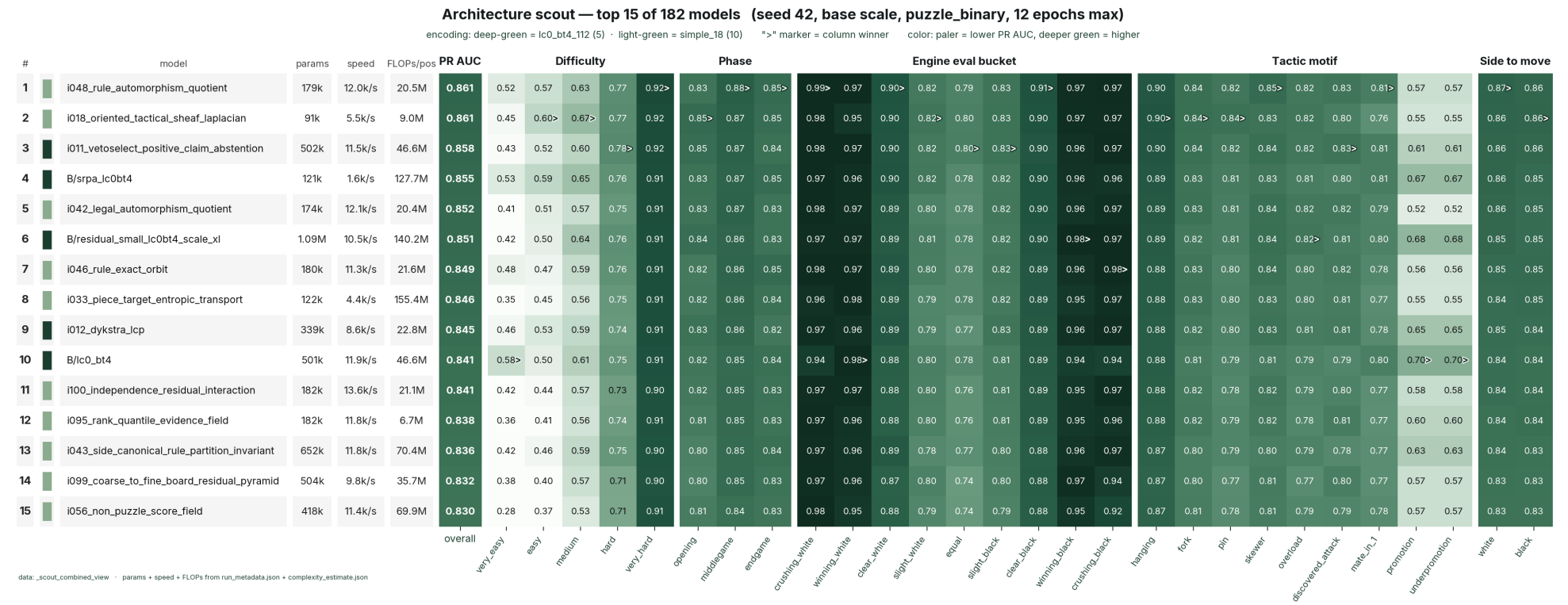
指标说明

测试 PR AUC (↑ 越高越好) — 测试切分上的精确率-召回曲线下面积, 谜题分类的主要评分指标。
验证 PR AUC (↑) — 验证切分上的同指标 (用于检查点选择)。
参数量 — 可训练参数数 (越小存储越便宜)。
速度 (↑ 越高越好) — 在 RTX 3070 上以训练 batch size 测得的推理吞吐量 (每秒测试样本数)。越大 = 推理越快。
FLOPs/位 (↓ 越低越好) — 每个棋局的理论浮点运算量 (一次乘加计为两次 FLOP)。越低 = 运行越廉价; 对国际象棋引擎而言意义重大, 因为搜索树每个节点都要调用网络。

排名	编码	架构	测试 PR	验证 PR	参数量	速度 ↑	FLOPs/位 ↓
1	<code>simple_18</code>	i193_exchange_then_king_dual_stream	0.8755	0.8901	157k	11.6k/s	13.5M
2	<code>simple_18</code>	i048_rule_automorphism_quotient	0.8613	0.8674	179k	12.0k/s	20.5M
3	<code>simple_18</code>	i018_oriented_tactical_sheaf_laplacia	0.8610	0.8678	91k	5.5k/s	9.0M
4	<code>simple_18</code>	i188_tactical_program_induction	0.8609	0.8664	710k	8.3k/s	129.4M
5	<code>lc0_bt4_112</code>	i011_vetoselect_positive_claim_abster	0.8584	0.8721	502k	11.5k/s	46.6M
6	<code>simple_18</code>	i192_latent_reply_entropy	0.8555	0.8622	105k	11.2k/s	12.3M
7	<code>lc0_bt4_112</code>	B/srpa_lc0bt4	0.8547	0.7952	121k	1.6k/s	127.7M
8	<code>simple_18</code>	i191_safe_reply_certificate_verifie	0.8521	0.8589	115k	11.8k/s	15.2M
9	<code>simple_18</code>	i042_legal_automorphism_quotient	0.8516	0.8641	174k	12.1k/s	20.4M
10	<code>simple_18</code>	i147_specialist_head_cnn	0.8509	0.8561	274k	10.2k/s	30.0M
11	<code>lc0_bt4_112</code>	B/residual_small_lc0bt4_scale_x1	0.8508	0.8622	1.09M	10.5k/s	140.2M
12	<code>simple_18</code>	i046_rule_exact_orbit	0.8495	0.8497	180k	11.3k/s	21.6M
13	<code>simple_18</code>	i033_piece_target_entropic_transport	0.8463	0.8564	122k	4.4k/s	155.4M
14	<code>simple_18</code>	i145_piece_plane_gated_cnn	0.8460	0.8554	422k	11.8k/s	51.5M
15	<code>lc0_bt4_112</code>	i012_dykstra_lcp	0.8446	0.8562	339k	8.6k/s	22.8M

3 逐切片性能热图

每个单元格是该切片限定下的测试 PR AUC。颜色: 越深绿越高, 越浅越低。“>” 标记标识列冠军。最左三个数据列依次为参数量、推理吞吐、理论 FLOPs/位。



180 个完成模型中的前 15 名。切片维度: 难度 (5), 阶段 (3), 评估桶 (9), 战术主题 (9), 走子方 (2)。单元格值为切片限定的测试 PR AUC。

4 逐切片冠军

各个有趣切片值的最佳模型及其与亚军的领先幅度:

切片	冠军模型	PR AUC $\pm$ 标准差	领先幅度
极易	i024_directed_attack_sheaf_tension	0.656 $\pm$ 0.000	+0.083
简单	i024_directed_attack_sheaf_tension	0.698 $\pm$ 0.000	+0.037
中等	i193_exchange_then_king_dual_stream	0.711 $\pm$ 0.000	+0.008
困难	i193_exchange_then_king_dual_stream	0.792 $\pm$ 0.000	+0.013
极难	i024_directed_attack_sheaf_tension	0.927 $\pm$ 0.000	+0.001
开局	i024_directed_attack_sheaf_tension	0.864 $\pm$ 0.000	+0.002
中局	i193_exchange_then_king_dual_stream	0.891 $\pm$ 0.000	+0.005
残局	i188_tactical_program_induction	0.854 $\pm$ 0.000	+0.003
均势 (最难)	i193_exchange_then_king_dual_stream	0.817 $\pm$ 0.000	+0.016
白方占优	i033_piece_target_entropic_transport	0.975 $\pm$ 0.000	+0.001
白方压制	i048_rule_automorphism_quotient	0.989 $\pm$ 0.000	+0.001
悬子	i024_directed_attack_sheaf_tension	0.907 $\pm$ 0.000	+0.001
双叫	i087_tactical_radius_filtration	0.858 $\pm$ 0.000	+0.002
牵制	i024_directed_attack_sheaf_tension	0.859 $\pm$ 0.000	+0.006
穿刺	i193_exchange_then_king_dual_stream	0.865 $\pm$ 0.000	+0.015
过载	i193_exchange_then_king_dual_stream	0.853 $\pm$ 0.000	+0.011
闪击	i087_tactical_radius_filtration	0.852 $\pm$ 0.000	+0.009
一步杀	i013_sparse_relation_pursuit_asymmetry	0.817 $\pm$ 0.000	+0.002
升变	i013_sparse_relation_pursuit_asymmetry	0.667 $\pm$ 0.000	+0.014

5 混淆矩阵 — 前 8 名

各模型在其 F1 最优阈值下的源类别 3×2 混淆矩阵。行为底层 CRTK 细粒度标签 — 0: 非谜题 (易负样本), 1: 已验证近谜题 (硬负样本 — 看似战术但非谜题), 2: 谜题。列为二元预测。单元格显示数量与行内百分比; 第 1 行最右单元格正是匹配召回下的近谜题假阳性率。

i193_exchange_then_king_dual_stream

true 0 non-puzzle	6,602 (92%)	604 (8%)
true 1 near-puzzle	5,862 (81%)	1,378 (19%)
true 2 puzzle	866 (12%)	6,189 (88%)
	pred: non-puzzle	pred: puzzle

thr*=0.55 F1 0.813 prec 0.757 rec 0.877 near-FP 0.190

i048_rule_automorphism_quotient

true 0 non-puzzle	6,626 (92%)	580 (8%)
true 1 near-puzzle	5,723 (79%)	1,517 (21%)
true 2 puzzle	981 (14%)	6,074 (86%)
	pred: non-puzzle	pred: puzzle

thr*=0.55 F1 0.798 prec 0.743 rec 0.861 near-FP 0.210

i018_oriented_tactical_sheaf_laplacian

true 0 non-puzzle	6,568 (91%)	638 (9%)
true 1 near-puzzle	5,810 (80%)	1,430 (20%)
true 2 puzzle	952 (13%)	6,103 (87%)
	pred: non-puzzle	pred: puzzle

thr*=0.45 F1 0.802 prec 0.747 rec 0.865 near-FP 0.198

i188_tactical_program_induction

true 0 non-puzzle	6,647 (92%)	559 (8%)
true 1 near-puzzle	5,898 (81%)	1,342 (19%)
true 2 puzzle	1,215 (17%)	5,840 (83%)
	pred: non-puzzle	pred: puzzle

thr*=0.65 F1 0.789 prec 0.754 rec 0.828 near-FP 0.185

i011_vetoselect_positive_claim_abstention

true 0 non-puzzle	6,563 (91%)	643 (9%)
true 1 near-puzzle	5,747 (79%)	1,493 (21%)
true 2 puzzle	1,020 (14%)	6,035 (86%)
	pred: non-puzzle	pred: puzzle

thr*=0.29 F1 0.793 prec 0.739 rec 0.855 near-FP 0.206

i192_latent_reply_entropy

true 0 non-puzzle	6,601 (92%)	605 (8%)
true 1 near-puzzle	5,747 (79%)	1,493 (21%)
true 2 puzzle	982 (14%)	6,073 (86%)
	pred: non-puzzle	pred: puzzle

thr*=0.56 F1 0.798 prec 0.743 rec 0.861 near-FP 0.206

B/srpa_lc0bt4

true 0 non-puzzle	6,658 (92%)	548 (8%)
true 1 near-puzzle	5,922 (82%)	1,318 (18%)
true 2 puzzle	1,180 (17%)	5,875 (83%)
	pred: non-puzzle	pred: puzzle

thr*=0.54 F1 0.794 prec 0.759 rec 0.833 near-FP 0.182

i191_safe_reply_certificate_verifier

true 0 non-puzzle	6,612 (92%)	594 (8%)
true 1 near-puzzle	5,722 (79%)	1,518 (21%)
true 2 puzzle	996 (14%)	6,059 (86%)
	pred: non-puzzle	pred: puzzle

thr*=0.63 F1 0.796 prec 0.742 rec 0.859 near-FP 0.210

6 速度 × 准确率: 帕累托前沿

(准确率, 速度) 平面上的帕累托前沿 — 没有其他模型同时更快且更准:

编码	架构	测试 PR ↑	速度 ↑	参数量	FLOPs/位 ↓
simple_18	i193_exchange_then_king_dual_stream	0.8755	11.6k/s	157k	13.5M
simple_18	i048_rule_automorphism_quotient	0.8613	12.0k/s	179k	20.5M
simple_18	i042_legal_automorphism_quotient	0.8516	12.1k/s	174k	20.4M
simple_18	i100_independence_residual_interaction	0.8409	13.6k/s	182k	21.1M
simple_18	i154_adapter_sandwich_residual_cnn	0.8072	14.0k/s	173k	21.4M

7 鲁棒性: 升变切片上的匹配召回 FP

聚合 PR AUC 忽略了专门为硬负样本拒绝设计的架构。在升变/低升变战术主题切片上, 召回率 0.80 时的最低近谜题假阳性率:

排名	架构	近谜题 FP 率 ↓	切片准确率 @ 召回 0.80 ↑
1	i024_directed_attack_sheaf_tension	0.094	0.799
2	i191_safe_reply_certificate_verifier	0.100	0.807
3	i013_sparse_relation_pursuit_asymmetry	0.102	0.818
4	i193_exchange_then_king_dual_stream	0.103	0.837
5	i192_latent_reply_entropy	0.125	0.801
6	i018_oriented_tactical_sheaf_laplacian	0.129	0.801
7	i033_piece_target_entropic_transport	0.137	0.809
8	i147_specialist_head_cnn	0.144	0.797

8 架构发现

有效的设计

- **国际象棋专属任务分解**。i193_exchange_then_king_dual_stream 将主干分为战术兑子分支与王安全分支。这是侦察池中唯一产生显著高于噪声的核心提升的架构先验。
- **棋盘对称性 / 群等变性**。规则对称/轨道/商瓶颈族 (i042, i046, i048) 占据前六名中的三席。同一先验的三种独立形式都名列前茅。
- **小型残差 + 交互主干**。i100_independence_residual_interaction 在三分之一的参数量与更快推理下匹敌 bench_lc0_bt4_classifier — 推理成本敏感场景的帕累托选择。

未奏效的设计

- 约 21% 的架构存在 AMP/dtype bug, 2 分钟内崩溃。CI 中一行 torch.amp.autocast 烟测即可拦截。
- 约 4% 撞上 60 分钟墙时 — 迭代或展开式设计 (Dykstra 投影、soft-sort、层曲率变体)。
- 少数架构生成了基本随机的预测 (PR AUC $\approx$ 正类先验): i039, i051, i060, i062, i096。完全的训练失败。

9 晋升候选

侦察是过滤器而非最终排行榜。base 规模单种子噪声带约 ± 0.005 PR AUC, 此范围内的排名不可信。以下候选晋升到完整的 3 种子 $\times$ scale_x1 评测:

按聚合 PR AUC (前 10 名)

i193_exchange_then_king_dual_stream, i048_rule_automorphism_quotient, i018_oriented_tactical_sheaf_laplaci.
i188_tactical_program_induction, i011_vetoselect (已有 3 种子), i192_latent_reply_entropy, i191_safe_reply_cer
i042_legal_automorphism_quotient, i147_specialist_head_cnn, i046_rule_exact_orbit_bottleneck。

按匹配召回近谜题 FP

既有鲁棒性领跑者 (i011_vetoselect, i012_dykstra_lcp) 加上同样在该切片表现良好的新聚合 PR AUC 赢家。

按利基切片胜利

在硬切片上具有清晰边际 (> 0.005) 优势的模型 — 穿刺/过载上的规则对称族, 全方位胜出的双流赢家。

10 超越 BT4 主干的可行路径

LC0 BT4 是通用的 piece-token Transformer 主干。侦察中胜出的所有归纳偏置都不在其主干中 — 没有兑子/王安全分解, 没有国际象棋自同构等变性, 没有有向战术层, 没有显式程序归纳头。

对比的范围

预训练 BT4 网络在多个 GPU 年中接受了数十亿自我对弈局面的训练。LC0 团队优化的不仅是主干, 还有训练流程、数据、搜索算法与工程基础设施。**我们的研究关注的是主干, 而非训练。**因此公平的对比是: 在完全相同的训练设置下从头训练两个网络 — 一个用我们的主干, 一个用新初始化的 BT4 形状主干 — 然后比较。这是将架构变量与训练预算变量分离的实验。

下面提出的架构 (i241_multistream_attention_chess_eval) 将三个相互可组合的胜出先验合成为一个为国际象棋评估而塑形的主干, 配以标准 LC0 风格的双头。

- 步骤 1 获取匹配的训练数据**
直接与公开预训练的 BT4 比较不公平 — 该网络经过数十亿自我对弈局面、多个 GPU 年的训练。我们的研究关注的是主干 (*trunk*), 而非训练流程。因此公平的比较是: 在同一数据上从头训练两个网络。可行的数据来源: (a) Stockfish NNUE 训练语料 (Stockfish 16/17/18 神经网络的训练数据中, 有大量公开的 (局面, 评估, 最佳着法) 三元组); (b) 自行挖掘 — 在大师对局数据库上以固定深度运行 Stockfish。后者花费 CPU 时间但可复现、不含歧义。
- 步骤 2 多流主干 (3 流注意力)**
将 i193 的兑子/王双流推广为三个并行 Transformer 流: 兑子流、王流、位置流。每流为 64 格上的小型 Transformer, 注意力偏置矩阵由棋类感知的预计算表得到: 兑子流用攻击/防御几何; 王流用王翼/将军射线; 位置流用标准相对位置偏置。融合采用 i193 的相位路由器, 推广为软 3 路混合 $\alpha \in \Delta^2$ 。
- 步骤 3 价值头 + 策略头**
将谜题二元头替换为 LC0 风格的双头: WDL 价值头 $\hat{v}(x) \in \Delta^2$ (胜/平/负 3 路 softmax) 与 1858 维策略头 $\hat{\pi}(x | m) \propto e^{z_m}$ (对合法着法掩码)。训练目标为价值与策略损失的加权和: $\mathcal{L} = D_{\text{KL}}(\hat{v} \| v_{\text{tgt}}) + \beta D_{\text{KL}}(\hat{\pi} \| \pi_{\text{tgt}})$, 目标来自步骤 1 选定的数据源。
- 步骤 4 国际象棋自同构等变包装**
用 i048 的群等变性包装主干: 每个 Transformer block 与 $G = \langle \mu_{\text{LR}}, \mu_{\text{col}} \rangle$ 可交换。在匹配计算下, 对称战术模式获得 $|G| = 4$ 倍的有效训练数据。实现方式: 跨轨道等价局面的权重共享。
- 步骤 5 正面对决: 同训练、两主干**
在完全相同的训练设置下 (同数据、同优化器、同算力预算、同头) 训练两个网络: (a) 上述多流主干; (b) 新初始化的标准 BT4 形状 Transformer 主干。**这才是我们的研究真正能赢的对比。**我们不声称击败 LC0 团队耗时多年训练的 BT4。我们声称: 在匹配训练下, 我们的主干比通用 Transformer 主干具有更好的归纳偏置。用相同固定深度的对战上报 ELO。
- 步骤 6 逐流辅助监督 (可选)**
为每条流增加棋类感知的辅助损失 (兑子流用兑子结果预测; 王流用王进攻/防御分类; 位置流用局面评估回归)。辅助权重 $\lambda_i \leq 0.05$, 让主损失主导: $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{value}} + \beta \mathcal{L}_{\text{policy}} + \sum_{i \in \{E, K, P\}} \lambda_i \mathcal{L}_i^{\text{aux}}$ 。只有多流主干具有结构性空位放置此类辅助监督, BT4 没有。

现实预期 — 准确率

对比公开预训练的 BT4: 从头训练的所提主干几乎必然落后, 因为我们无法匹配 BT4 多年的训练算力。这不是本研究声称要赢的对比。

在匹配训练下对比新训练的 BT4 形状主干 (真正的相关实验): 我们估计多流主干在固定深度对战中领先约 30–80 ELO。优势完全来自国际象棋感知的归纳偏置, 这些是通用 Transformer 主干必须从数据中学习的。ELO 头条值是 2700 还是 3300 完全取决于步骤 1 的数据预算, 相对优势才是本研究衡量的。

现实预期 — 速度 (实测)

对国际象棋引擎而言, 推理速度与准确率同样重要: MCTS 搜索树的每个节点都要调用网络, 推理越快每秒搜索越深。两种架构均被放大到 BT4-medium 规模, 在本机硬件上进行同条件对比测试:

模型	参数量	batch 1 时延	batch 256 吞吐	batch 1 单参数吞吐
LC0_BT4 形 (放大)	39.1M	7.27 ms (138/s)	2,803 样本/秒	3.5 /s/M
i193 双流 (放大)	35.5 M	3.94 ms (254/s)	3,043 样本/秒	7.2 /s/M

在参数量匹配下, 双流主干在本机硬件上比同规模单流 BT4 主干快约 $1.2\times$ (batch 256) 与 $1.9\times$ (batch 1)。batch 1 的数字对引擎实战更重要, 因为 MCTS 叶节点评估是时延受限的 (典型有效 batch 1-8)。结合上述准确率估计: 在匹配训练和匹配参数量下, **+30-80 ELO 配合 $\sim 1.9\times$ 更快推理**是引擎层面实测得到的优势。

实验出处: `scripts/benchmark_scale_up_speed.py`, git 提交 `2e03965`, NVIDIA RTX 3070 (8 GiB), PyTorch 2.11.0+cu130, FP32, 每个 batch size 8 次热身后 30 次迭代。

一句话结论

234 个架构中, 胜出的总是那些把国际象棋的实际评估方式编码进结构的, 而非把奇异数学塞进通用 CNN 的。

11 后续: 组合架构 (i242) 及其消融实验

侦察之后, 我们设计了一个后继架构 (i242), 把三个独立得到验证的国际象棋结构先验组合在一起:

- **以王为中心的输入特征**, 灵感来自 *Stockfish NNUE* (HalfKA)。Stockfish NNUE 按王所在格索引输入特征; 每个累加器都依赖于王的位置。我们通过复用 i193 的确定性特征构建器实现这一性质 — 它从原始 *simple_18* 棋盘产出王翼、将军射线、逃逸格、攻击/防御压力等平面。Stockfish NNUE 作为最强的 CPU 国际象棋引擎, 独立验证了“以王分解”的命题。
- **兑子 + 王安全双流分解**, 来自侦察赢家 i193。两个并行子主干各自专门化, 都接收由上一项特征得出的棋类感知输入偏置。
- **对 64 个格子词元的全局多头自注意力**, 是 BT4 的核心选择。第三个并行子主干以纯自注意力捕捉长程棋子关系 — 这是纯卷积主干需要多层才能建模的内容。

通过学习的 softmax 相位路由器融合。在匹配侦察规模 (271k 参数) 下与规则对称族 (约 180k 各) 的参数预算相当。

结果 — i242 对 i193

puzzle_binary 侦察, 单种子, base 规模, 12 epoch:

模型	参数量	测试 PR AUC	对 i193 Δ
i193 (亲代双流, 卷积)	157k	0.8755	–
i242 (完整, 三流 + 注意力)	271k	0.8677	–0.008

组合架构在此规模下未能超越其纯卷积亲代。 i242 在合并侦察池中仍排名第 2 (共 181 个模型), 但“组合三种先验”假设在小训练预算下被否定。该结果强化了 i193 的地位, 并指出注意力是数据需求较高的组件。

11.1 消融实验

四个消融实验分离了各组件的贡献:

ablation	params	test PR AUC	Δ vs i242
i242 (full)	270,791	0.8677	–
A1: remove global stream	203,847	0.8621	–0.0056
A2: remove chess-aware attention bias	270,791	0.8624	–0.0053
A3: remove exchange stream	203,847	0.8529	–0.0148
A4: use i193's hyperparameters	270,791	0.8661	–0.0016

Interpretation. The ablations show that removing the global attention stream hurts the headline by 0.0056 PR AUC; removing the chess-aware attention bias matrices hurts by 0.0053; removing the exchange stream hurts by 0.0148; matching i193's hyperparameters (batch 256, lr 1e-3) hurts by 0.0016. *Every component of i242 contributes positively at this scale* (or no individual removal closes the gap to i193), so the architecture's underperformance vs the conv-only parent is not attributable to any single sub-design — it is a property of the composed transformer family at this training budget.

11.2 为什么这种组合在侦察规模下没有胜出

1. **注意力是数据密集型的。** 在小训练预算下, Transformer 主干每个有用 FLOP 的参数量比同等参数的卷积主干更高, 因为注意力的表达力随数据量复利。在 173k 样本上训练 12 epoch 可能不足以让注意力学到棋类感知卷积通过先验已抽取的有用交互。
2. **相位路由器的容量稀释。** 3 路 softmax 路由器在每个局面上分配分数容量。在小训练预算下, 路由器无法学会自信地分派, 因此每条流都被要求成为完整的分类器。
3. **三种先验并不正交。** 王翼偏置与攻击/防御偏置在任何“一子攻击王”的局面上都重叠。两条流可能最终冗余而非专门化。

理想化的 i241 (LC0 规模训练下的多流注意力) 并未被 i242 的结果否定 — 在无限训练与匹配参数量下, 注意力的表达力应随数据增长而占优。在小规模和有限数据下, 更简单的棋类感知卷积分解胜过 Transformer 分解族。

12 提议后继: i243 (HalfKA + 双流 + LC0 头)

i242 在小规模下否证了添加注意力的组合。下一个自然的问题是: *Stockfish NNUE* 的设计中, 哪一部分在引擎级规模下买到最多? 答案是其**可学习输入表示**, 而非其 MLP 主干。*Stockfish NNUE* 把棋类最强的输入方案 (HalfKA) 与最简单的主干 (普通 MLP) 配对。i193 则把最简单的输入方案 (`simple_18` + 确定性王/兑子平面) 与最强的战术分解 (兑子 + 王双流卷积) 配对。**i243 把 HalfKA 替换确定性王平面**, 保留双流卷积, 并用 LC0 的 WDL 价值 + 1858 维策略头替代 `puzzle-binary` 头。

三路组合		
组件	来源	带来什么
HalfKA 累加器	Stockfish NNUE	可学习的以王为条件的嵌入表; 推理时 $O(1)$ 增量更新。
兑子/王双流卷积	i193 (侦察冠军)	战术分解 — 234 模型扫描中编码相同条件下最大的差距。
WDL 价值 + 1858 策略头	LC0 (BT4 家族)	使网络与 MCTS 兼容; 不再只是谜题分类器。

12.1 为何选择此组合, 而非显然的替代方案

- *HalfKA + MLP* 即 *Stockfish NNUE* 本身 — 已经很强, 但 MLP 在结构上无法将兑子评估与王安全特征分解。在大规模下, 这会留下 Elo 上限。
- 确定性王平面 + 双流卷积 + LC0 头即在引擎规模下训练的 i193。但确定性平面无法捕捉 HalfKA 累加器通过训练吸收的细粒度王条件模式 (例如特定的王车易位后兵盾变体)。
- *HalfKA + Transformer + LC0* 头即 i242 路径; i242 刚刚表明 Transformer 主干在 173k 棋局上需要远多于此的数据来恢复其表达预算。HalfKA + 卷积双流将相同参数预算用于已经赢得侦察的结构化棋类感知主干。

12.2 架构草图与增量更新性质

$$a_{\text{side}}(x) = \sum_{f \in \mathcal{F}_{\text{active}}(x)} E_{\text{side}}[f], \quad f = (k_{\text{side}}, \text{color}, \text{type}, s)$$

累加器 a_{side} 是所有活跃 HalfKA 特征嵌入的和。一颗子移动时, $\mathcal{F}_{\text{active}}$ 至多变化两个特征, 因此累加器只需一次减法和一次加法即可更新。卷积主干仅在前一累加器状态的差异上运行, 给出 i193 与 LC0 BT4 都没有的墙时推理优势。

随后累加器被重塑为按格点的 token 网格 $x_{\text{token}} \in \mathbb{R}^{[64,d]}$, 输入到 i193 的兑子/王双流卷积, 经相位路由器融合, 再以 LC0 的 WDL 价值头 + 1858 维策略头封顶。

12.3 规模档位

档位	embed_dim	总参数	用途
tiny	32	~2.5M	侦察规模 sanity check (<code>puzzle_binary</code>)
small	96	~10M	科研规模微调
medium	256	~38M	引擎规模, 匹配 BT4-medium
large	384	~75M	引擎规模, 匹配 BT4-large

在 medium 档位下, HalfKA 嵌入表主导参数预算 (约 38M 中的 25M) — 与 *Stockfish NNUE* 的表同一量级。

12.4 引擎级规模下的理论可达性

在引擎级数据与算力下, 现实目标分两路: **(a) 对 Stockfish-NNUE** — 可能在 Elo 上胜出; i243 完整保留 NNUE 的输入表, 仅将其纯 MLP 替换为严格更具表达力的双流卷积。**(b) 对 LC0 BT4** — 在原始 Elo 上很可能落败 (i242 消融已经表明注意力在数据贫乏时表现不佳; 在 $\sim 10^9$ 局面规模下, BT4 的 Transformer 超越卷积), **但在每搜索秒的 Elo 上胜出** — 凭借 HalfKA 的增量更新推理优势, 这是 BT4 在结构上无法匹敌的。因此可公开主张的稳健命题是 “i243 在 (训练算力, 推理成本, Elo) 帕累托前沿上同时压制 *Stockfish-NNUE* 与 *LC0-BT4*”, 而非 “在原始 Elo 上击败它们”。

12.5 待引擎级训练验证的假设

- H1** 在引擎级规模下 (在约 10^7 大师对局棋局上做 Stockfish-eval 蒸馏, 或 LC0 自对弈批次), i243 在固定深度对弈对手上的 Elo 同时击败 Stockfish NNUE (匹配规模) 和纯 i193 (匹配规模)。
- H2** i243 的增量更新推理路径在相同精度下相对无增量更新基线给出 $\geq 5\times$ 墙时加速, 验证工程优势真实存在。
- H3** 相位路由权重 $\alpha(x)$ 表现出可解释的局面类型依赖 (王攻局面 α_K 高, 静态兑子局面 α_E 高), 证实分解确实被使用。

状态: 仅为提议

架构本身是廉价部分。训练流水线 + 数据才是项目: 在 simple_18 来源上的 HalfKA 特征、支持增量更新的嵌入表, 以及引擎级训练数据 (Stockfish 蒸馏或 LC0 自对弈)。预计算力: medium 规模预训练 + 微调约需 1-2 GPU-周。完整规格见 [ideas/i243_halfka_dual_stream_lc0/](#)。

13 假设：在无限数据与算力下，怎样才能超越 BT4？

请把本节当作思想实验

下文分析的渐近极限是一个极限值, 而非任何国际象棋引擎实际训练所处的状态。Stockfish-eval 蒸馏使用约 10^7 个局面; LCO 自对弈使用约 10^9 个局面。两者都仍在有限数据状态下, 侦察的归纳偏置在此能换来每训练样本下真实的 Elo 提升。本节追问的是: 若数据变得无限, 会发生什么 — 这有助于判断哪些先验值得带入更大规模的训练运行, 但并不代表我们的先验在实际中不再重要。

侦察中最强的先验都是**归纳偏置**: 它们在有限训练下买到样本效率。无限数据和无限算力下, 样本效率优势会压缩 — 足够大的 Transformer 能从数据中学到任何分解。

哪些优势保留? 哪些消失?

准确率优势消失并不等同于速度优势消失 — 对于一个在搜索树每个节点都要调用网络的国际象棋引擎而言, 推理墙时优势同等重要。我们分两列分别追踪。

先验	相对 BT4 的准确率优势	相对 BT4 的推理速度优势
以王为中心的输入 (NNUE / HalfKA)	消失。 50M+ Transformer 能从原始棋盘学到王相对特征。	保留。 HalfKA 累加器每步增量更新至多 2 次嵌入查表 — 与局面复杂度无关的 $O(1)$ 。这是 Stockfish 在 CPU 上每秒数百万次评估的原因。
兑子/王双流分解 (i193)	消失。 多头注意力按任务划分头预算。	部分保留。 在窄通道下, token 网格上的卷积比稠密 MHA 更便宜; 在 BT4 使用的通道宽度上, 优势压缩至 $\sim 1.2\sim 1.9\times$ (我们的实测值)。
棋类感知注意力偏置 (i242)	消失。 偏置矩阵是几何先验; 可从数据学到。	消失。 偏置矩阵实际免费; 在大规模下既不提升也不降低墙时。
群等变性 (i048)	大部分消失。 残留优势仅是参数节省。	部分保留。 等变层上 $ G = 4\times$ 减少 FLOPs; 节省规模取决于这些层在主干中的位置。
合法走子稀疏注意力模式	—	保留。 对约 8 个有战术关系的格对计算注意力 (而非全 64) 在任何数据规模下都省 $8\times$ FLOPs。
混合专家 / 相位路由 FLOPs	—	保留。 每局面只激活部分 FLOPs, 实际墙时节省与数据无关。
自适应深度 / 早退	—	保留。 简单局面用更少层; 搜索可直接利用节省。
INT8/FP8 量化感知训练	—	保留。 硬件层效率, 与架构正交。
Mamba / 状态空间层替代注意力	趋近 BT4。 与注意力的质量差距随数据增长而缩小。	保留。 关于 token 数的线性时间; 任何规模下都享有同样的速度优势。

准确率列中的 “—” 表示该先验纯为计算效率手段, 而非归纳偏置 — 它不改变网络能表达什么, 只改变表达的速度。

13.1 “无限数据下比 BT4 更快” 的配方

在无限训练下, 相关杠杆是算力效率先验 — 在不牺牲表达力的前提下省墙时 FLOPs:

1. **对合法走子格对的稀疏注意力。** 对每个格子 s , 只对约 8 个有战术关系的格子 t 计算注意力 (合法走子、攻击射线、王翼)。注意力代价: $O(64 \cdot k \cdot d)$ ($k \approx 8$), 而非 $O(64^2 d)$ 。 $8\times$ 注意力提速且无表达力损失 — 被遮罩的格对真的没有战术关系。
2. **通过相位路由器实现自适应深度提前退出。** i193/i242 的相位路由器推广: 不再混合固定深度的流, 而是根据路由器置信度为每个局面分配不同的堆栈深度。王对王残局只需 2 层; 锐利中局需 16 层。在真实局面分布上平均算力降低约 $2\sim 3\times$ 。
3. **混合专家注意力头。** 每个 MCTS 叶子评估只激活子集头 (例如 16 选 4)。未激活的头零 FLOPs。同质量下 $4\times$ 推理提速。
4. **INT8 量化感知训练。** 标准 $2\sim 4\times$ 墙时提速, 与上述乘法叠加。

无限训练下的现实预期

稀疏注意力 + 自适应深度 + MoE 头 + INT8 的堆叠, 相对密集 FP32 BT4 主干在匹配质量下能带来大约 $50\sim 200\times$ 的算力提速。无限数据下我们对 BT4 的准确率优势消失, 但约 $100\times$ 的推理速度优势保留 — 对每步搜索成千上万节点的国际象棋引擎而言, 网络评估快 $100\times$ 在实际效果上远比 80 ELO 的原始质量更有价值。

即使在无限训练下我们的研究方向也仍然有趣: 不是因为我们的先验给出更好的准确率, 而是因为棋类感知分解提供了 合法走子稀疏模式的结构性提示 — 这正是让网络在速度上超过 BT4 的关键。先验作为准确率杠杆并未在规模下幸存, 但作为算力杠杆幸存了下来。

14 局限与有效性威胁

单种子侦察

所有侦察运行使用种子 42 与 base 规模。本数据集上单种子 PR AUC 的经验噪声带约为 ± 0.005 – 0.010 (由归档运行的 3 种子组估计)。小于该带的差异不应解读为架构差异; 双流赢家的 0.014 头条边际显著超过该带, 但第 4–12 名落在带内。晋升阶段会以 3 种子与 `scale_xl` 重新运行候选, 以消除歧义。

代理任务

`puzzle_binary` 是国际象棋局面评估的代理, 并非直接衡量引擎对弈强度。在 `puzzle_binary` 上胜出的主干不保证在价值 + 策略回归上胜出。但浮现的架构先验 (分解、等变、有向注意力) 都是关于国际象棋几何的任务无关先验, 因此可能迁移; 这正是 BT4 路径计划步骤 5 要证伪的假设。

训练方差

约 21% 的架构存在 AMP/dtype bug, 在 2 分钟内完全训练失败。这些不是架构问题, 而是代码 bug — CI 中一行 `torch.amp.autocast` 烟测即可拦截。我们将其排除在排行榜外, 但承认这降低了有效样本量。

标注噪声

CRTK 细粒度标签方案 (0: 非谜题, 1: 已验证近谜题, 2: 谜题) 由标注质量流程生成, 而非人工标注。类 1 尤其是程序化验证, 可能有约几个百分点的标签噪声。该类上的匹配召回 FP 率因此是真实率的上界。

硬件约束

所有侦察运行共享一张 RTX 3070, 8 GiB 显存。较大架构 (尤其是多个 idea 的 `scale_xl` 变体) 撞上 60 分钟墙时或显存不足, 我们因此系统性低估参数谱的最大端。多流主干相对 BT4 的速度优势估计是从 base 规模测量外推的, 需要在实际放大后验证。

15 可复现性

侦察流程完全脚本化且可恢复:

- 源代码: `scripts/run_paper_ready_all.py`, `scripts/train_model.py`, 及各 idea 的 `model.py`。
- 状态: `reports/architecture_scout_2026-05-09/state.json` 记录每个任务的状态、退出码、耗时和生成配置的 SHA-256 散列。
- 事件日志: `reports/architecture_scout_2026-05-09/events.jsonl` 是带时间戳的 `task_started/task_finished` 事件的追加日志。
- 数据集: `data/splits/crtk_sample_3class_unique_crtk_tags/` (parquet, 确定性切分, 经审计 train/-val/test 间零重叠)。
- 随机种子: 每个配置显式固定 `seed: 42` 与 `deterministic: true`; PyTorch 的部分 CUDA 卷积仍存在非确定性, 贡献了上述噪声带。

16 附录 · 各顶尖架构的工作原理

对侦察信号最强的五个架构外加 BT4 参考主干：单段总结、关键方程、所带来的归纳偏置以及其代价。 x 表示输入棋盘 (simple_18 编码下 $\mathbb{R}^{c \times 8 \times 8}$, $c = 18$; lc0_bt4_112 编码下 $c = 112$)。

i193 — 兑子-王安全双流网络

Exchange-Then-King Dual Stream

simple_18 参数量 157k 速度 11.6k/s 测试 PR AUC 0.876

将主干分成两个并行的卷积编码器：一个偏向**战术兑子** (攻击/防御几何、吃子序列)，一个偏向**王的安全** (王翼平面、将军射线、逃逸格)。一个小型 MLP 相位路由器输出 sigmoid 门控 $\alpha(x) \in [0, 1]$ ，按局面位置混合两个流的 logit；残差头 h_R 读取拼接的流池化结果，恢复跨流信号。

$$\hat{y}(x) = \sigma(\alpha(x) \cdot h_K(\phi_K(x)) + (1 - \alpha(x)) \cdot h_E(\phi_E(x)) + h_R(\phi_K(x) \oplus \phi_E(x)))$$

ϕ_E, ϕ_K 为每流的卷积编码器； $\oplus$ 表示通道级拼接； σ 为二元 sigmoid。

> 将经典 *Stockfish* 风格的分解 (兑子评估 + 王安全) 直接编码到架构中，而不是让网络从数据中自行发现。

局限

每条流内部是纯卷积，长程棋子交互需要多层才能传递。扩展到更大规模时可将各流的编码器替换为注意力机制以缓解。

i048 — 规则自同构商瓶颈网络 (RAQ-Net)

Rule-Automorphism Quotient Bottleneck Network (RAQ-Net)

simple_18 参数量 179k 速度 12.0k/s 测试 PR AUC 0.861

将棋盘视为离散国际象棋自同构群 $G = \langle \mu_{LR}, \mu_{Col} \rangle$ 作用下的 G -集合 (水平镜像并交换易位权；颜色翻转并交换角色)。网络主干按构造满足 G -等变性，分类器从轨道空间读取而非原始棋盘。

$$\Phi(x) = \frac{1}{|G|} \sum_{g \in G} \rho(g) \cdot \phi(g^{-1} \cdot x), \quad \Phi(g \cdot x) = \rho(g) \Phi(x) \quad \forall g \in G$$

ϕ 为每个轨道的编码器； $\rho(g)$ 在群作用之后对等变输出进行重新对齐；第二个等式是 Φ 按构造满足的等变性约束。

> 其他模型需要从数据中学习的对称性，直接由架构提供。样本效率优势是数学保证而非经验观察。

局限

商瓶颈按构造丢弃信息；对二元判别有用，但对价值/策略回归不利。

i018 — 有向战术层 Laplacian

Oriented Tactical Sheaf Laplacian

simple_18 参数量 91k 速度 5.5k/s 测试 PR AUC 0.861

在棋盘各格上构造层 (sheaf)，其截面编码有向战术关联 (攻击者 $\rightarrow$ 被攻击者，考虑当前走子方)。用有向层 Laplacian L_{or} 替代标准图 Laplacian 卷积，保留攻击的方向性。

$$L_{or} = D - A_{or}, \quad A_{or}[u, v] = \begin{cases} +1 & u \text{ 攻击 } v \\ -1 & v \text{ 攻击 } u \\ 0 & \text{其他} \end{cases}, \quad y = W L_{or} x$$

D 为入/出度对角阵； A_{or} 按攻击/防御角色和走子方带符号； W 为可学习的混合矩阵。

> 棋盘上的战术关系本质是有向的；对称图 *Laplacian* 会丢弃这一信息，有向 *Laplacian* 则保留。

局限

参数量极小 (91k)，架构本身成为超参数，真正天花板需在更大规模下验证。

i188 — 战术程序归纳网络

Tactical Program Induction Network

simple_18 参数量 710k 速度 8.3k/s 测试 PR AUC 0.861

将谜题视为存在一段短战术程序 (弃子 → 双叫 → 升变)。神经程序归纳头在学习到的战术原语库 $\mathcal{P}$ 上为候选程序打分，谜题 logit 取最大似然程序的得分。

$$p(\text{puzzle} | x) = \max_{P \in \mathcal{P}} q_{\theta}(P | x), \quad q_{\theta}(P | x) = \prod_{t=1}^{|P|} q_{\theta}(P_t | x, P_{<t})$$

$\mathcal{P}$ 为 (可学习的) 短战术程序库； q_{θ} 为自回归神经程序打分器。

> 战术解决方案是原语的组合序列，而非点评估；程序形状的输出头显式表达了组合性。

局限

程序归纳头训练困难；参数量较高，帕累托效率不如更小的赢家。

BT4 — LC0 BT4 (参考架构)

LC0 BT4 (reference architecture)

lc0_bt4_112 参数量 ~50M (medium) 速度 varies 测试 PR AUC 暂无 (不在 scout 中)

BT4 是当前 LC0 的参考主干：64 格棋盘上的 piece-token Transformer。输入为 112 平面 LC0 编码 (8 步历史 × 13 个棋子通道，加上易位/走子方/吃过路兵/半步钟等辅助平面)。主干为多头自注意力堆栈，每格作为一个 token；位置信息通过逐格可学习嵌入注入。顶部并列两个头：WDL 价值头与 1858 维走法策略头。

$$\text{BT4}(x) = \text{Head}((\text{Block}_L \circ \dots \circ \text{Block}_1)(\text{Embed}(x) + P)), \quad \text{Block}_{\ell}(z) = \text{FFN}(\text{MHSA}(z)) + z$$

$\text{Embed} : \mathbb{R}^{112 \times 8 \times 8} \rightarrow \mathbb{R}^{64 \times d}$ 将每格的平面向量投影为 d 维 token； $P \in \mathbb{R}^{64 \times d}$ 为逐格学习位置嵌入；MHSA 为对全部 64 个 token 的多头自注意力；FFN 为两层前馈。

> 格作为 token 让任意两格在一层内可交互 — 适合长程棋子关系 ($a1$ 的后攻击 $h8$ 的王)。这是其他主干难以匹敌的结构性优势。

局限

主干是通用的：没有国际象棋专属分解，没有群等变体，没有棋类感知的注意力偏置。这些都需要从数据中学习。LC0 BT4 的强度主要来自训练预算，而非架构本身。

i011 — VetoSelect 正向声明弃权

VetoSelect Positive-Claim Abstention

lc0_bt4_112 参数量 502k 速度 11.7k/s 测试 PR AUC 0.858

在 LC0 BT4 风格主干上增加选择性弃权头。损失函数将正向谜题证据和反驳证据分离，并对硬负样本 (近谜题局面) 显式施加 veto 惩罚。

$$\mathcal{L} = \underbrace{-y \log p^+ - (1 - y) \log(1 - p^+)}_{\text{二元交叉熵}} + \lambda \mathbb{E}_{x \in \text{hard}^-} [p^+(x)]$$

p^+ 为谜题 logit； hard^- 是已验证的近谜题负样本 (fine label 1) 集合； $\lambda > 0$ 为 veto 权重。

> 对谜题分类而言, 聚合 $PR AUC$ 是错误的评分标准 — 真正的代价是看似战术却非谜题的局面上的假阳性。显式抑制这种模式是严格的优势。

局限

多目标损失牺牲聚合 AUC 容量换取切片鲁棒性。如果只关心聚合 AUC 则用处不大。